SIDOM

CHALLENGE TÉCNICO · DESARROLLO

Un embarque de COMEX

Un ejercicio corto para hacer en casa. Nos interesa cómo pensás el problema, no que llegues a una solución perfecta.

NODE.JS · TYPESCRIPT

El caso

Un caso simple y realista del dominio de **SIDOM**. No hace falta que sepas nada de comercio exterior: todo lo que necesitás está acá abajo.

Las cuatro entidades

Embarque	El envío internacional: origen, destino y estado.
Contenedor	Va dentro de un embarque · 1 embarque → N contenedores.
Ítem de carga	Qué va en cada contenedor · 1 contenedor → N ítems.
Documento aduanero	Los papeles del embarque. Son dos: la factura comercial y el packing list, que detalla qué bultos viajan y qué hay dentro de cada uno.

La tarea

Modelar las entidades y exponer una API que permita:

- Crear un embarque con sus contenedores.
- Agregar ítems a un contenedor.
- Consultar el estado de un embarque con todo su contenido.

Hasta dónde llevarlo es decisión tuya. Preferimos una API chica y bien pensada antes que una grande a medio terminar.

Qué entregar

Dos cosas, en un repositorio Git o Bitbucket al que podamos acceder.

1 · La API funcionando

Que se pueda levantar y probar siguiendo tus instrucciones. Si algo quedó a medias, dejalo dicho en el README en vez de disimularlo.

2 · Un README de una página

Es tan importante como el código. Cuatro puntos, breves:

- **Cómo levantarlo.** Los comandos, sin vueltas.
- **Decisiones de modelado.** Cómo representaste las entidades y sus relaciones, y por qué así.
- **Qué dejaste afuera y por qué.** Esto suma, no resta: recortar con criterio es parte del ejercicio.
- **Qué harías distinto con más tiempo.**

Una página alcanza. No buscamos documentación formal ni un diagrama prolijo: buscamos entender cómo pensaste el problema.

Reglas

> **Tiempo límite: 3 horas.** Es un límite real, no una sugerencia. Evaluamos lo que entra en ese tiempo, no que esté completo. Si te pasás, decilo en el README: es dato, no falta.

Una sola restricción técnica: Node.js y TypeScript. Es lo único que fijamos. El framework lo elegís vos: nuestro stack es NestJS, pero Express, Fastify o lo que uses habitualmente están igual de bien.

Todo lo demás lo decidís vos. Framework, base de datos o datos en memoria, ORM o SQL a mano, estructura de carpetas, tests o no, Docker o no. No hay una respuesta que estemos esperando.

Esto no es una trampa ni una omisión: las decisiones **son** el ejercicio. Resolver todo con datos en memoria y explicar por qué vale tanto como montar una base real. Lo que miramos es la coherencia entre lo que elegiste, el tiempo que tenías y lo que contás en el README.

Sobre el uso de IA

Podés usar las herramientas que uses habitualmente: Copilot, Cursor, Claude, las que sean. **Trabajamos con IA todos los días** y no tendría sentido pedirte que no la uses.

Lo único que pedimos es un párrafo en el README: qué generaste con IA, qué corregiste de lo que te devolvió y qué revisaste a mano.

Declararlo no resta. Lo que no funciona es entregar código que no puedas explicar: en la sesión de presentación se nota enseguida.

DESPUÉS DE LA ENTREGA

La sesión de presentación

Luego de que nos envíes el ejercicio, nos juntamos por videollamada. **No te vamos a pedir que programes en vivo:** la idea es que nos cuentes qué hiciste, cómo modelaste, cómo utilizaste la IA, y por qué tomaste las decisiones que tomaste.

No buscamos la respuesta perfecta: buscamos el porqué. Una decisión razonada y defendible nos dice más que la solución más elegante sin justificación.

Llegá con el repo abierto para recorrerlo juntos.